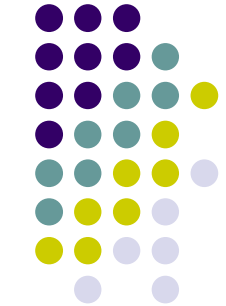
Today: Switch Scheduling

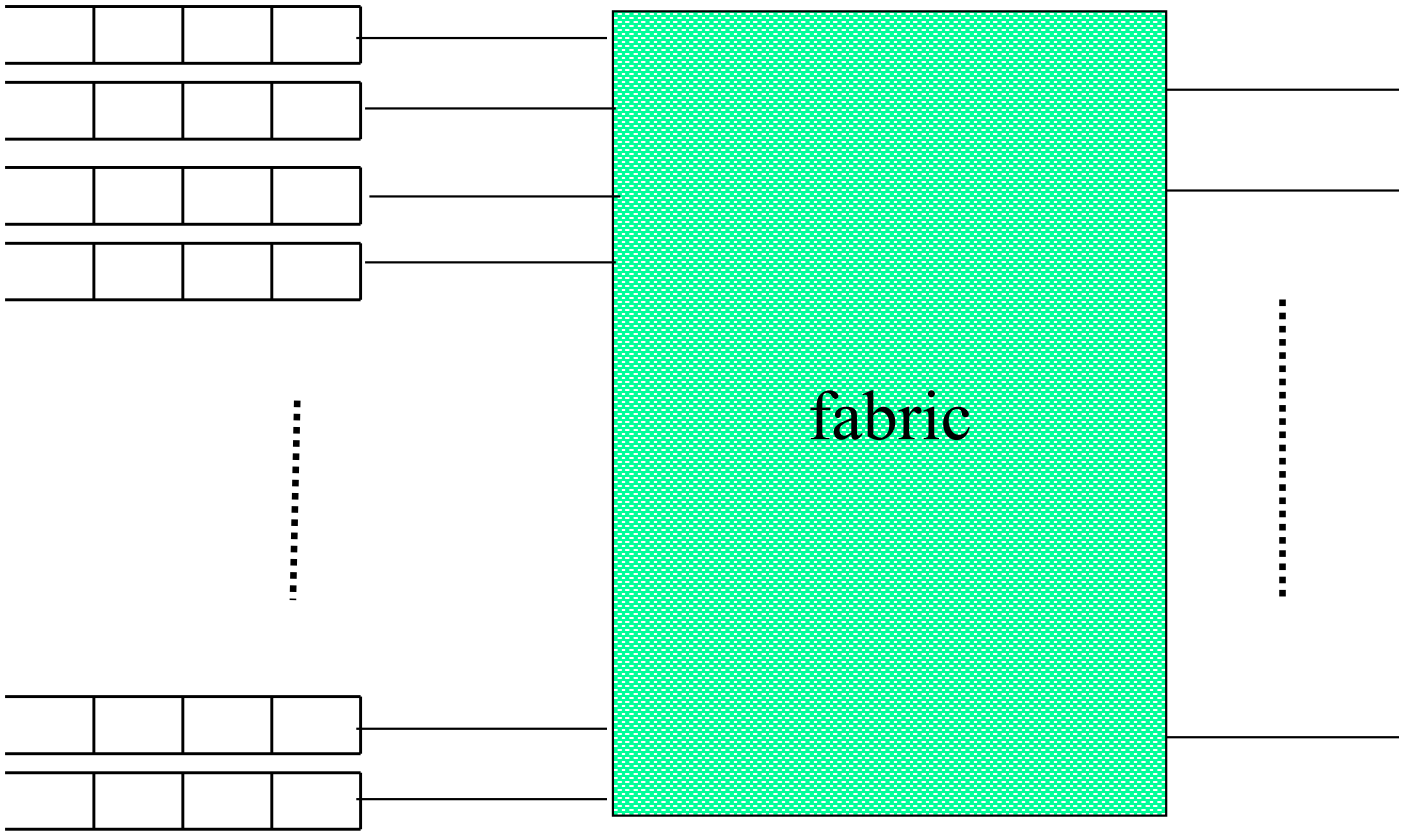


- Saw: how to deal with a single permutation
- Problem: How to deal with many-1 demand?
  - Buffering and scheduling strategies
  - General traffic model
  - Scheduling of a congested link

Switch: Simplified Model

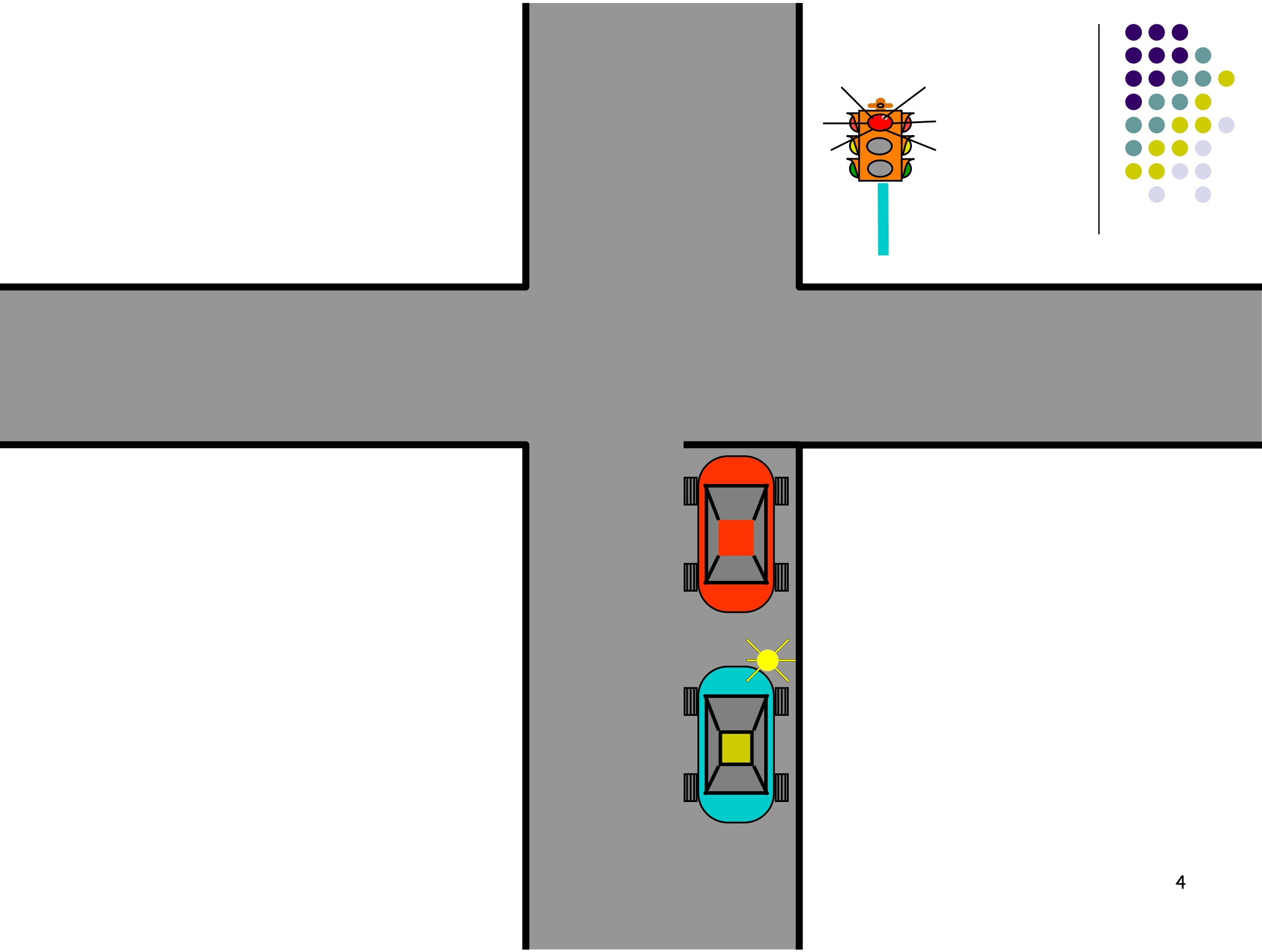
- $N \times N$ switch
- All packets have the same size: **cells.**
- All lines have the same speed:
  - Normalization: line speed = 1 cell/time slot

Input Queuing

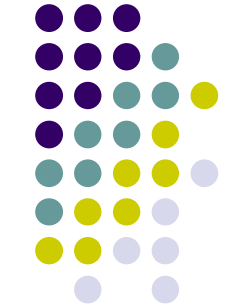


Fabric implements partial permutation, demand is arbitrary

➔ Packets wait in input buffers for fabric



Head of Line (HOL) Blocking



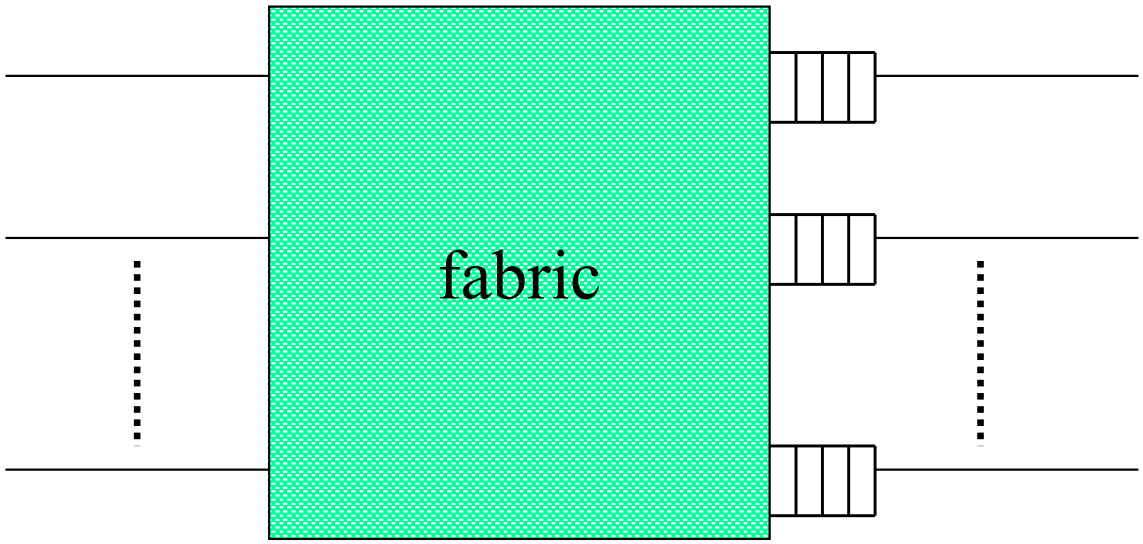
- When first packet can't move, all packets behind it are stuck even if their destination is free!
- Approximate calculation: suppose destinations are random. Then

$$E[\#\text{ distinct outputs}] = \sum_{i=1}^{n} \Pr[\text{some input has output } i]$$

Can show: 58.6%

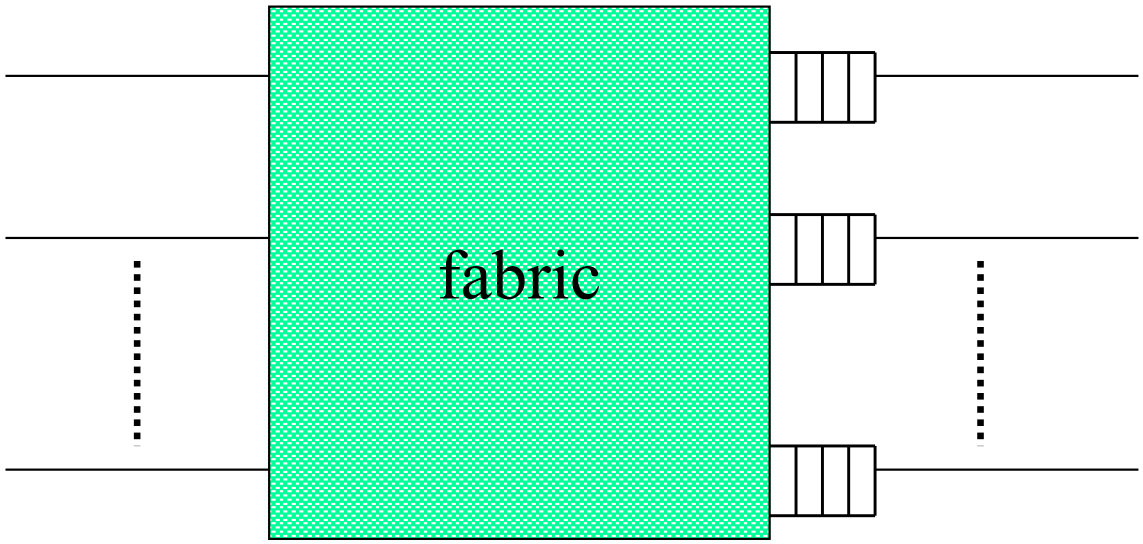
Output queuing

- Fabric takes a packet from **each input** in each line clock
  - Possibly more than one packet to an output port
- Packets wait for their link in **output** buffers

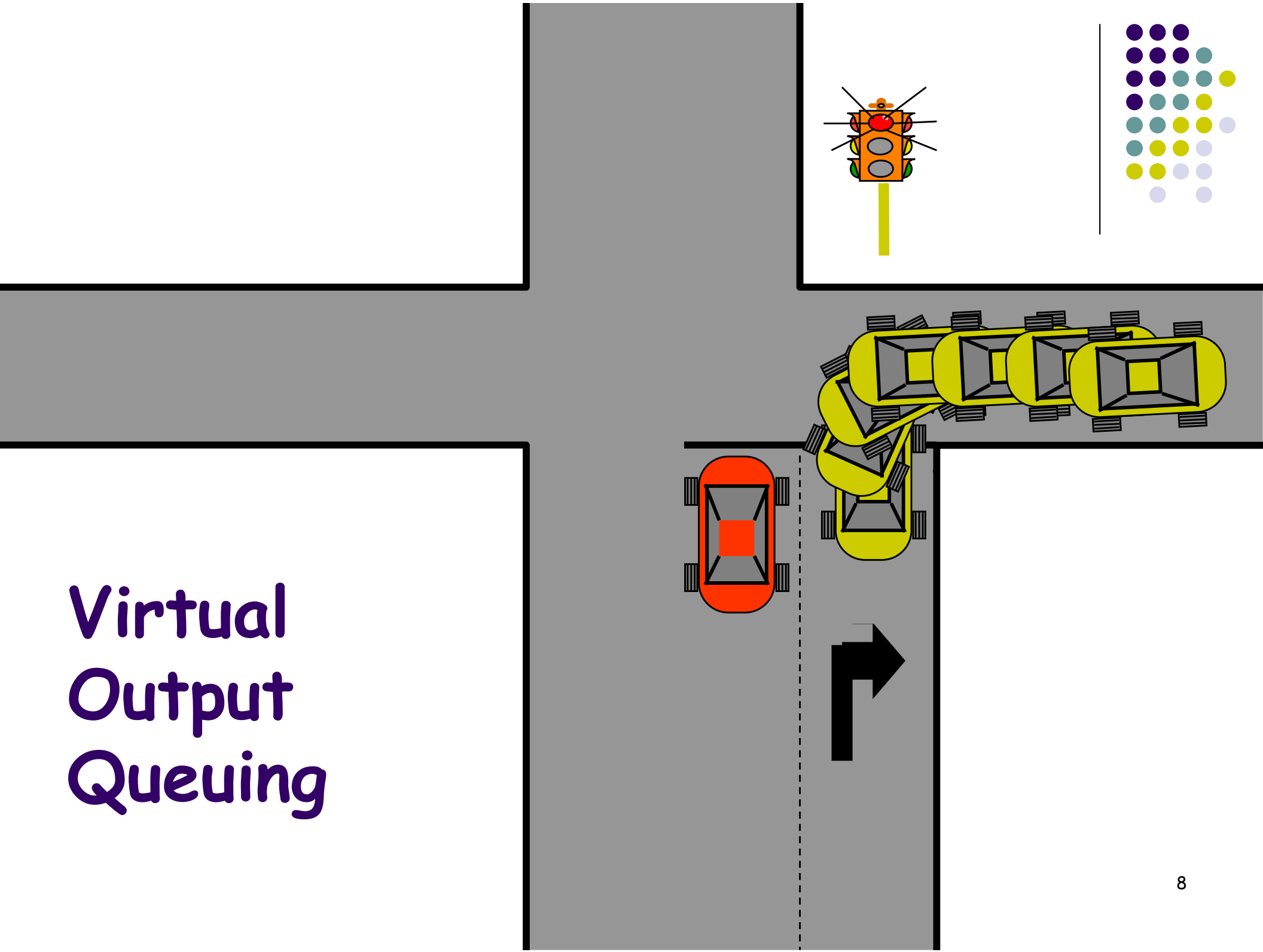


Output queuing

- No HOL blocking ➔ best possible throughput
  - Blocking only depends arrival pattern
- Can implement different queuing disciplines
- But: fabric must run N times faster than inputs...

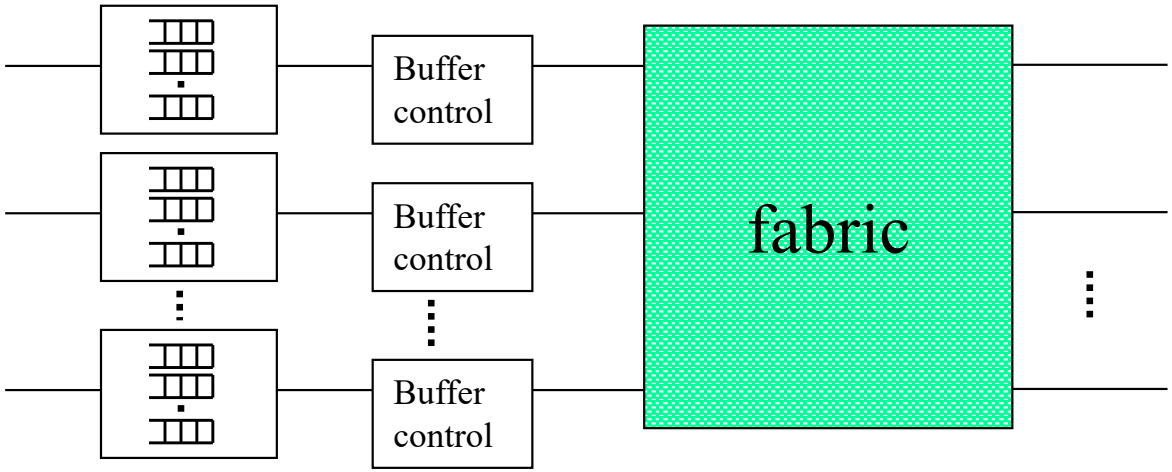


Virtual Output Queuing

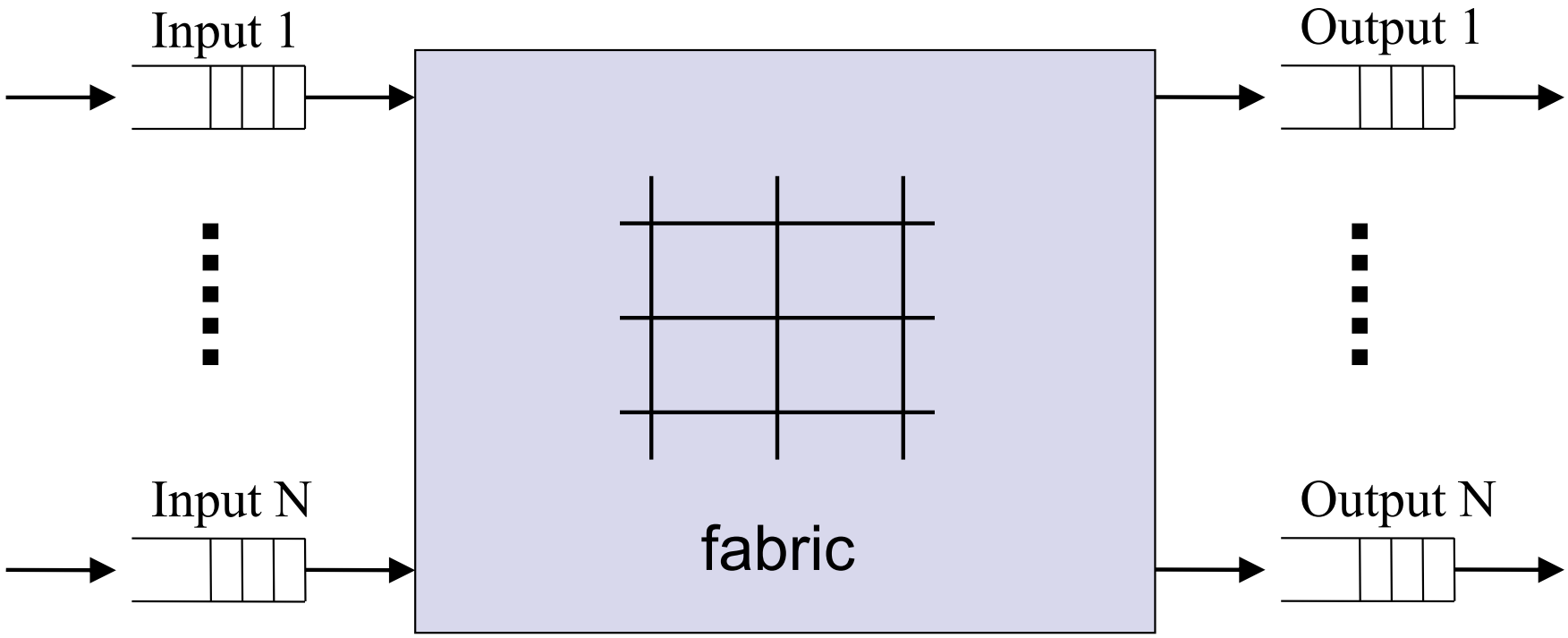


Virtual output queuing

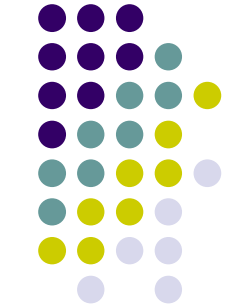
- In each input port: a queue for each output port (N^2 queues overall)
- Can get 100% utilization on traffic with random destinations
- Requires scheduling!



Combined Input Output Queuing (CIOQ) Switch

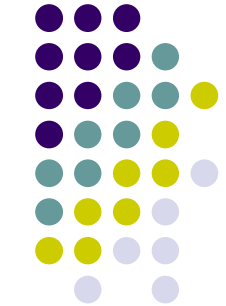


Combined Input Output Queuing (CIOQ) Switch



- Queues both in the input and in the output.
- For speedup of $1 < s < N$ buffering is required in both inputs and outputs.
- Can get **full emulation** of output-queued switch.

Mimic OQ with CIOQ switch



- We'll see that:

  A CIOQ switch with a speedup of 2 can behave identically to any OQ switch.

Problem statement

- Find the smallest speedup and the appropriate scheduling algorithm that:
  - Allows CIOQ to exactly mimic OQ, for any input pattern, and any output queue policy
  - Independent of switch size.
- 'Exactly mimic' means that under identical input the departure time of every cell from both switches is identical.

Scheduling Framework

Each time slot is broken into 4 phases:

1. Arrival Phase (input lines)
2. First scheduling phase (fabric)
3. Departure phase (output lines)
4. Second scheduling phase (fabric)

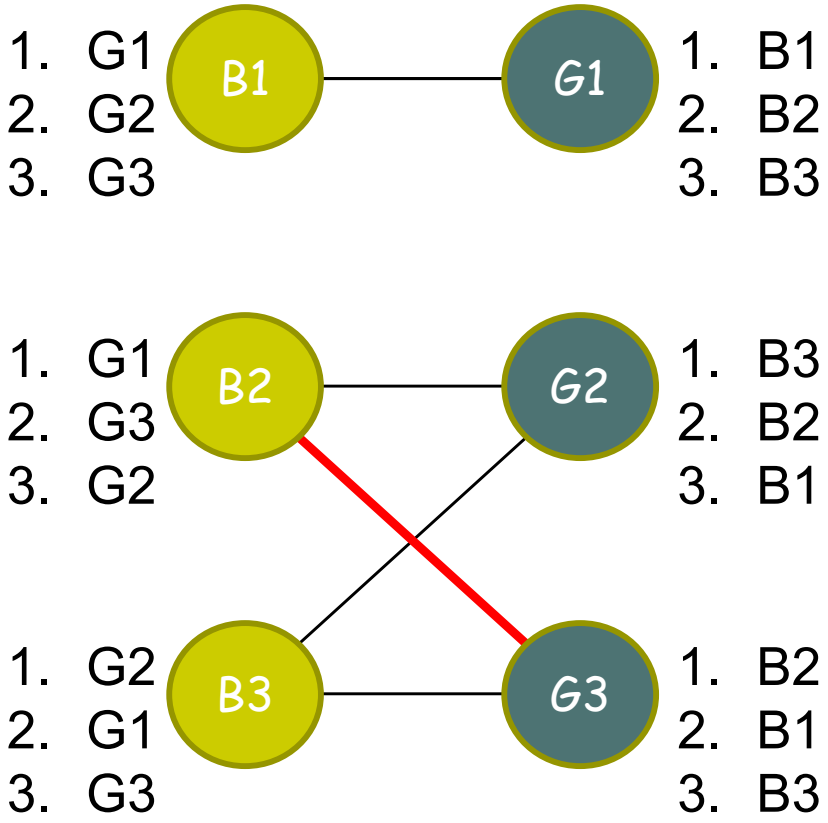
- Scheduling phase: algorithm chooses a set of cells that to send to output ports
  - We use the **stable marriage** algorithm to select the packets in each scheduling phase.

The stable marriage problem

- n boy
- Each , …, n
  - $r_g(b$) = 1 for b le
- Boys
- A matching (n couples) is **stable** if

there is no **blocking** pair: an unmarried pair, who both prefer each other to their spouses.

Example



- Matching:
B1-G1, B2-G2, B3-G3.
Stable?
- No! B2 - G3 blocking
- Matching:
B1-G1, B2-G3, B3-G2
Stable?
- Why?

Stable marriage algorithm

- Definitions:
  - M: partial matching
    - B_M, G_M: boys, girls matched in M
    - $M(b) = g$ if $(b, g) \in M$,
      $M(g) = b$ if $(b, g) \in M$.

Stable marriage algorithm

$M := \emptyset$.

Repeat

- Pick $g \in G - G_M$ (arbitrary).
- Find **first** b in g's list s.t. either:

  (i) $b \notin B_M$

  ➔ $M := M \cup \{(b, g)\}$.

  b is not matched yet

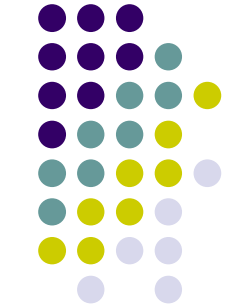
  (ii) $b \in B_M$ and $r_b(g) > r_b(M(b))$

  ➔ $M := (M \cup \{(b, g)\}) - \{(b, M(b))\}$

  b prefers g to his current match

Until $|M| = n$.

Stable marriage algorithm



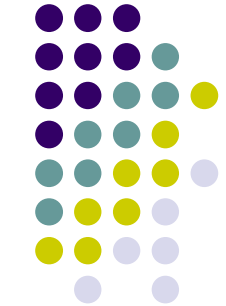
- Lemma 1: At all times, M is stable.
  - If $r_g(b) > r_g(M(g))$ then $b \in B_M$ and $r_b(g) < r_b(M(b))$.
- Lemma 2: n^2 iterations suffice.
  - Given a partial matching M, define the **potential**

$$\phi(M) = \sum_{b \in B_M} r_b\big(M(b)\big)$$

$\phi(M)$ increases by at least one unit in each iteration, and cannot exceed n^2.

QED

Input & output priority list



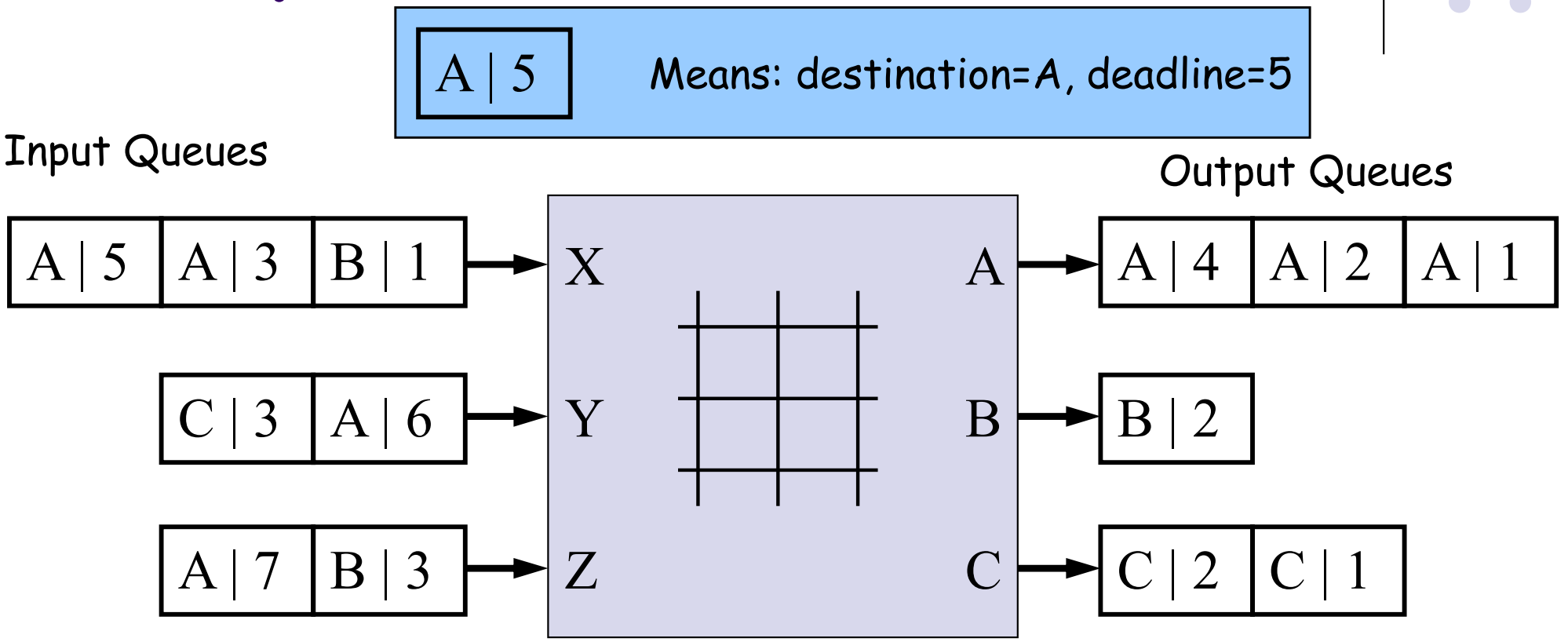
Ports rank their counterparts as follows:

- In an output port *A*, most important is the input port holding the cell for *A* with the closest deadline.

Inputs ports are more complicated:

- Consider a cell in input port X, going to output port *A* and deadline t.
  Its position in the queue of X is **1 + the number of packets in *A* with deadline before** t.

Example

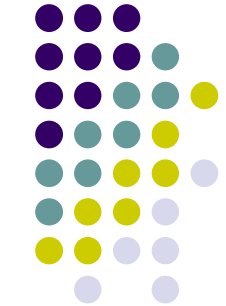


If cell [C|4] arrives in X, it is inserted at position 1+2, after [A|3] and pushing [A|5] one position back.

Input X prefers B, then A, then C

Output A prefers X, then Y, then Z

Speedup 2 is sufficient



Theorem: The algorithm can emulate any OQ switch, under any arrival pattern.

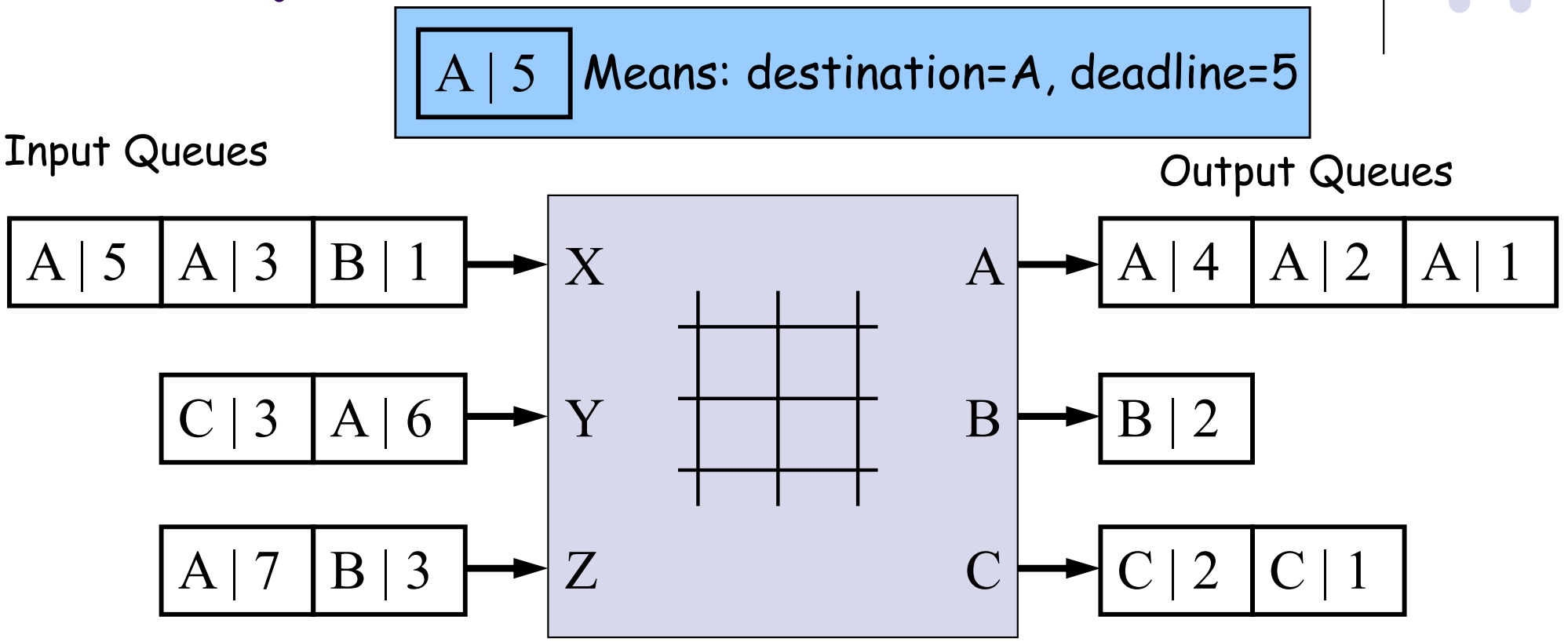
Definition: For a time slot t,

$L_t(c)$ **of cell** c with in = X and out = A **is**
#(cells in A with closer deadline) minus
#(cells preceding c in X)

Intuitively: how tight is the deadline of c.

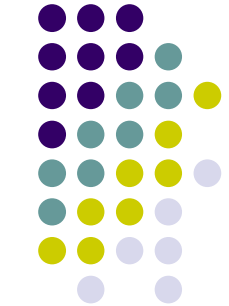
By construction: If cell c arrives at time t, then $L_t(c) = 0$.

Example



$$L(A|3) = 2 - 1 = 1$$
$$L(A|6) = 3 - 0 = 3$$

Speedup 2 is sufficient (cont.)



Lemma: For all $t \geq 0$, $L_{t+1}(c) \geq L_t(c)$.

Proof: $L(c)$ decreases by at most 2 in a time slot:

- In the arrival phase, one cell may arrive at input;
- In the departure phase, one cell may leave output.

- But c not scheduled by stable marriage algorithm, so in each scheduling phase either
  - #cells more urgent than c in its output increases by 1, or #cells ahead of it in its input is decreased by 1.

QED

Speedup 2 is sufficient

Proof (of theorem):

By Lemma, and since $L_0(c) = 0$, $L_{t(c)} \geq 0$ for all t.

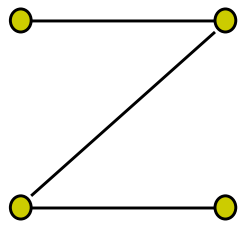
➔ upon deadline (start of time slot) of c, there's no one ahead of it in its input (if c is still there) and no one ahead of it in its output, so

- If c is still in its input, Stable Marriage Algorithm will transfer it to output in first scheduling phase.
- In any case, the output port will transmit c on departure phase.

QED

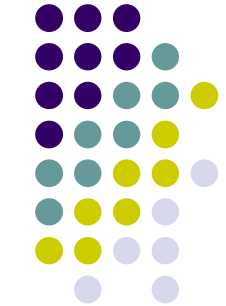
Practical Algorithms

- Objective: find a maximal matching between input ports and output ports
- Note: maximal is NOT maximum!
  - **Maximal** matching: a matching that cannot be extended. Size: at least half of maximum.



- but maximal $\geq \frac{1}{2}$ maximum (why?)

Practical Algorithm: PIM

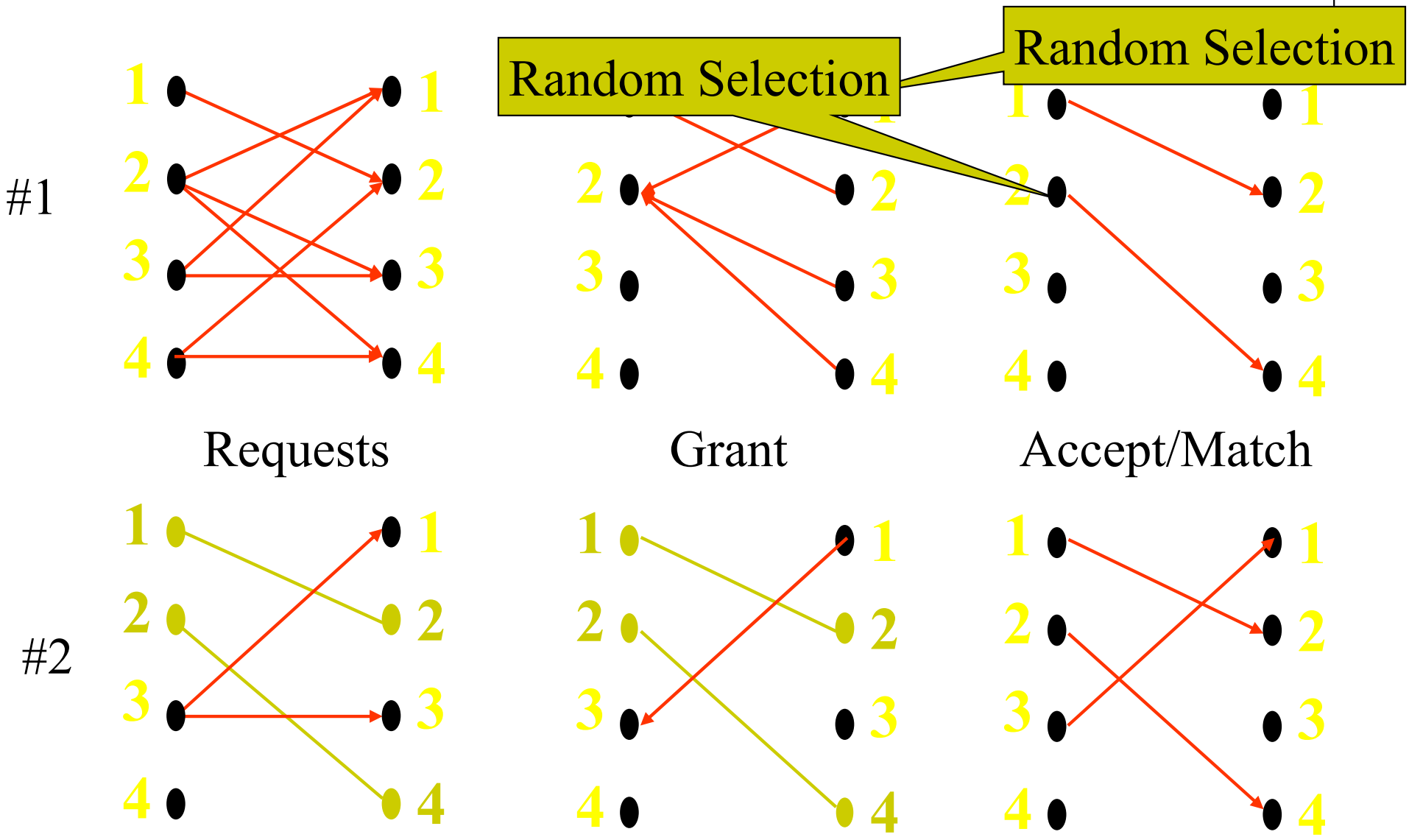


Parallel Iterative Matching

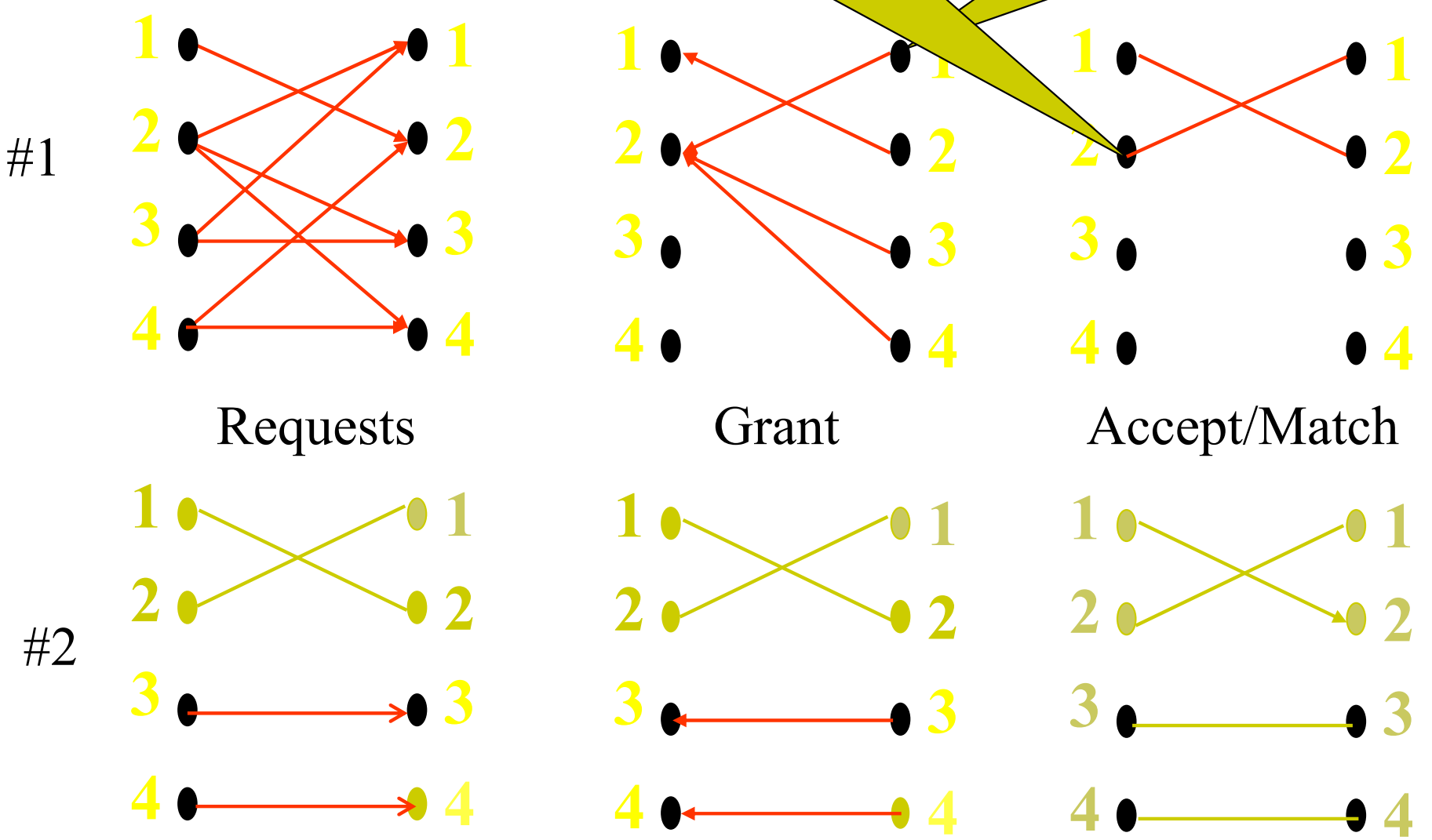
1. Each input sends REQUEST to all outputs it needs
2. Each output selects randomly one of the requesting inputs, send GRANT
3. Each input selects randomly one of the granting outputs, sends ACCEPT

$O(\log n)$ iterations guarantee maximal matching with high probability!

Parallel Iterative Matching



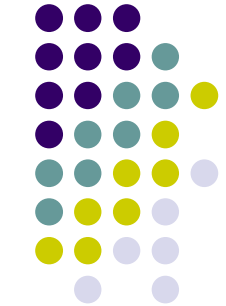
iSLIP


Round-Robin Selection
ound-Robin Selection
#1
1
2
3
4
Requests
Grant
Accept/Match
#2

iSLIP
Properties

- Looks random under low load
- TDM under high load
- Lowest priority to most recently matched
  - 1 iteration: fair to outputs
  - At most N iterations until request granted
- Implementation: N priority encoders
- Up to 100% throughput for uniform traffic

Summary



Implication: Speedup 2 is enough!

- But computational cost of algorithm is too high.

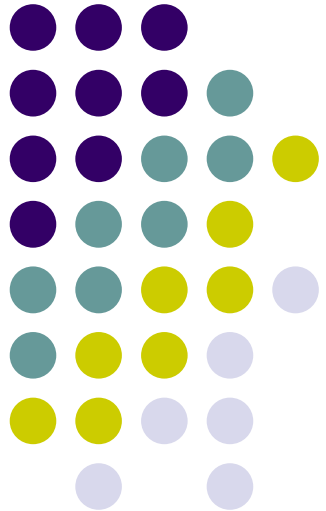
In practice, **maximal** matchings are good enough, with moderate speedup (3-4).

- Can be computed in linear time
- Can be approximated distributively (random, iSLIP)

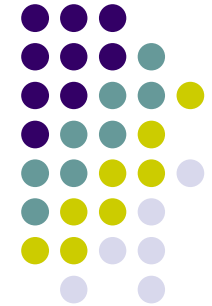
Where are we?

- At the network layer: switches
- Saw:
  - Sophisticated fabrics—good for partial permutations
  - Scheduling the fabric—because input is not permutation
- Now we'll look at scheduling of a congested link
- First—we move from packets to streams

Traffic analysis



A model for traffic streams



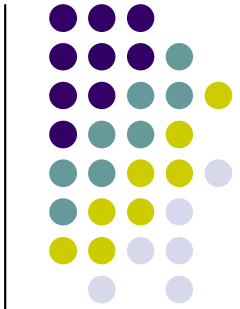
A **stream** (or **flow**) describes traffic at a given point over time.

Notation: For a stream f, $R^f(t)$ is the speed (in bps) at time t.

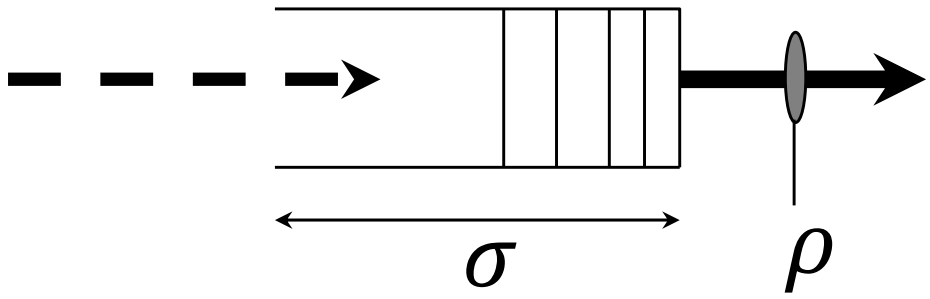
Total #bits in the stream in time $[x, y]$:

$$B^f(x, y) = \int_x^y R^f(t)dt$$

Network element: a buffer

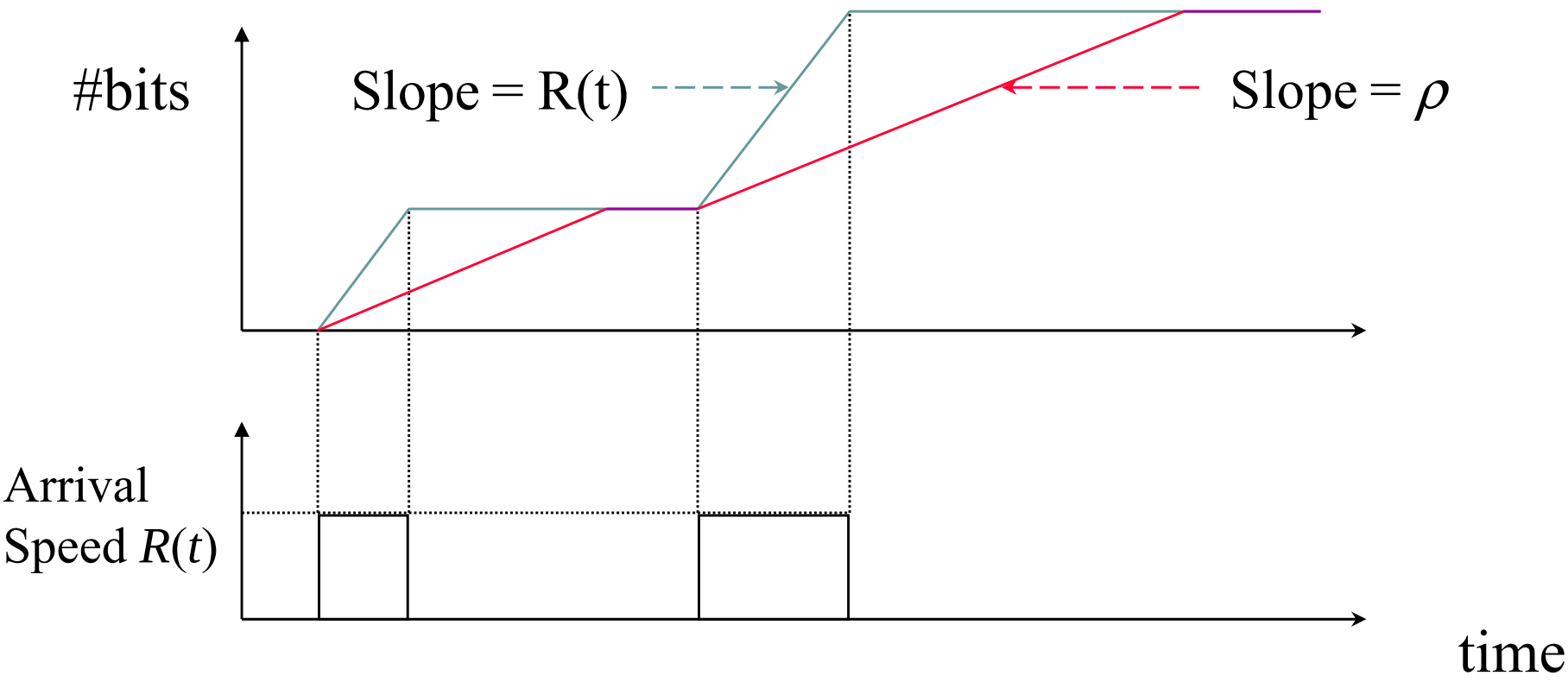


Consider a buffer of size σ and drain rate ρ.

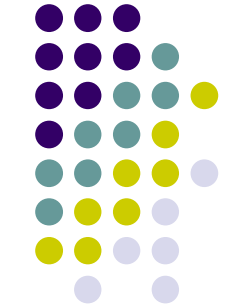


What streams can the buffer serve without losing data?

Example: Two packets arrive at a buffer

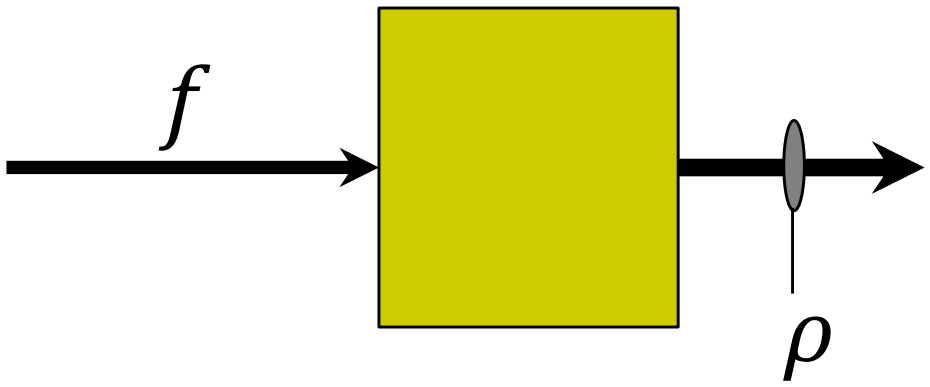


Definition of Backlog

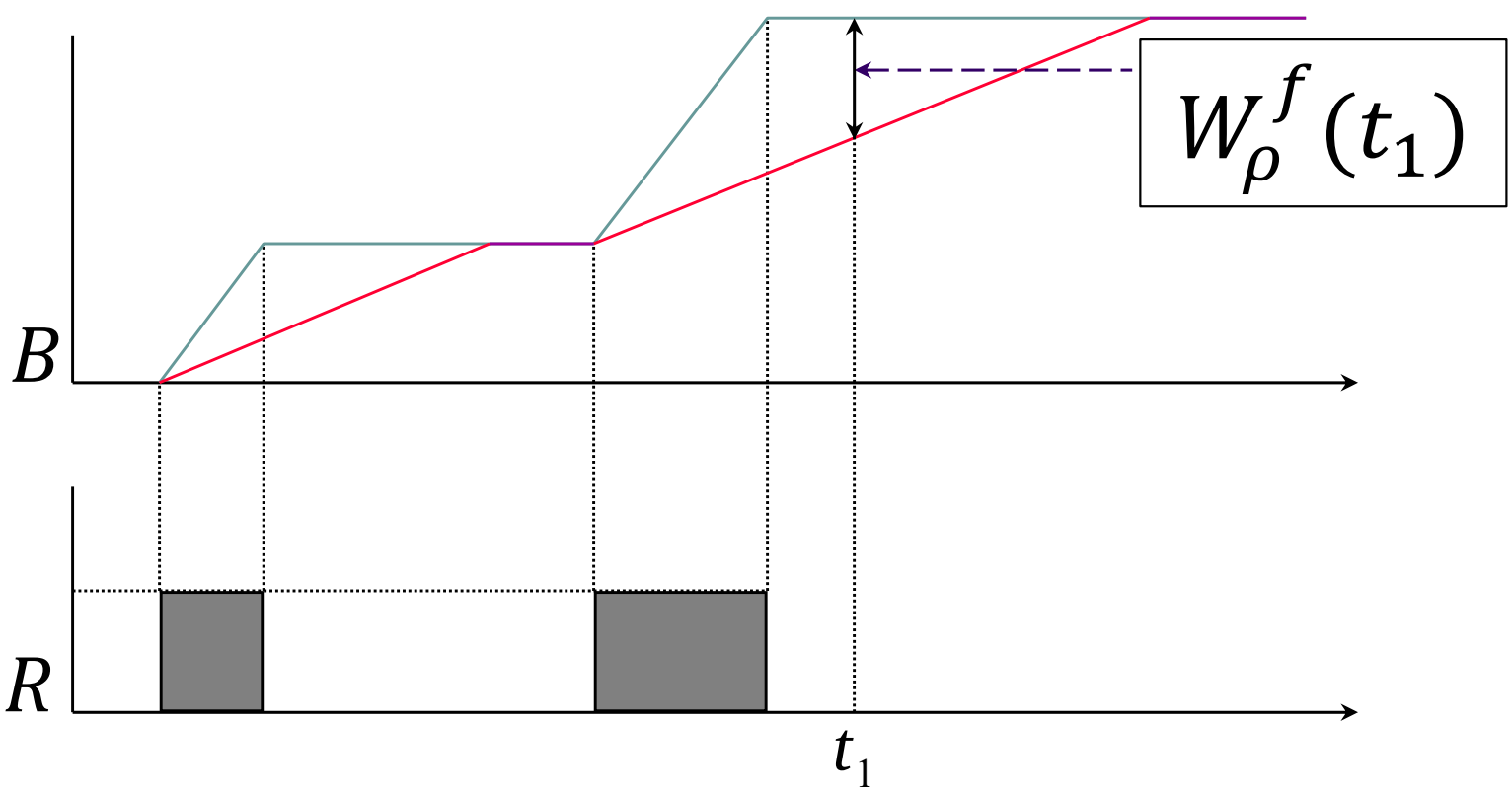


- A flow f with total bit function B^f, arrives at a device with drain rate ρ. The backlog at time t is:

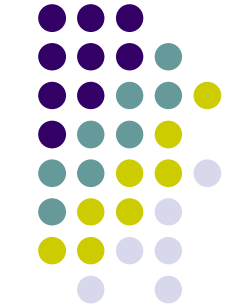
$$W_\rho^f(t) = \max_{s \le t}\{B^f(s,t) - \rho \cdot (t-s)\}$$



Graphical Representation



(σ, ρ)-constrained streams



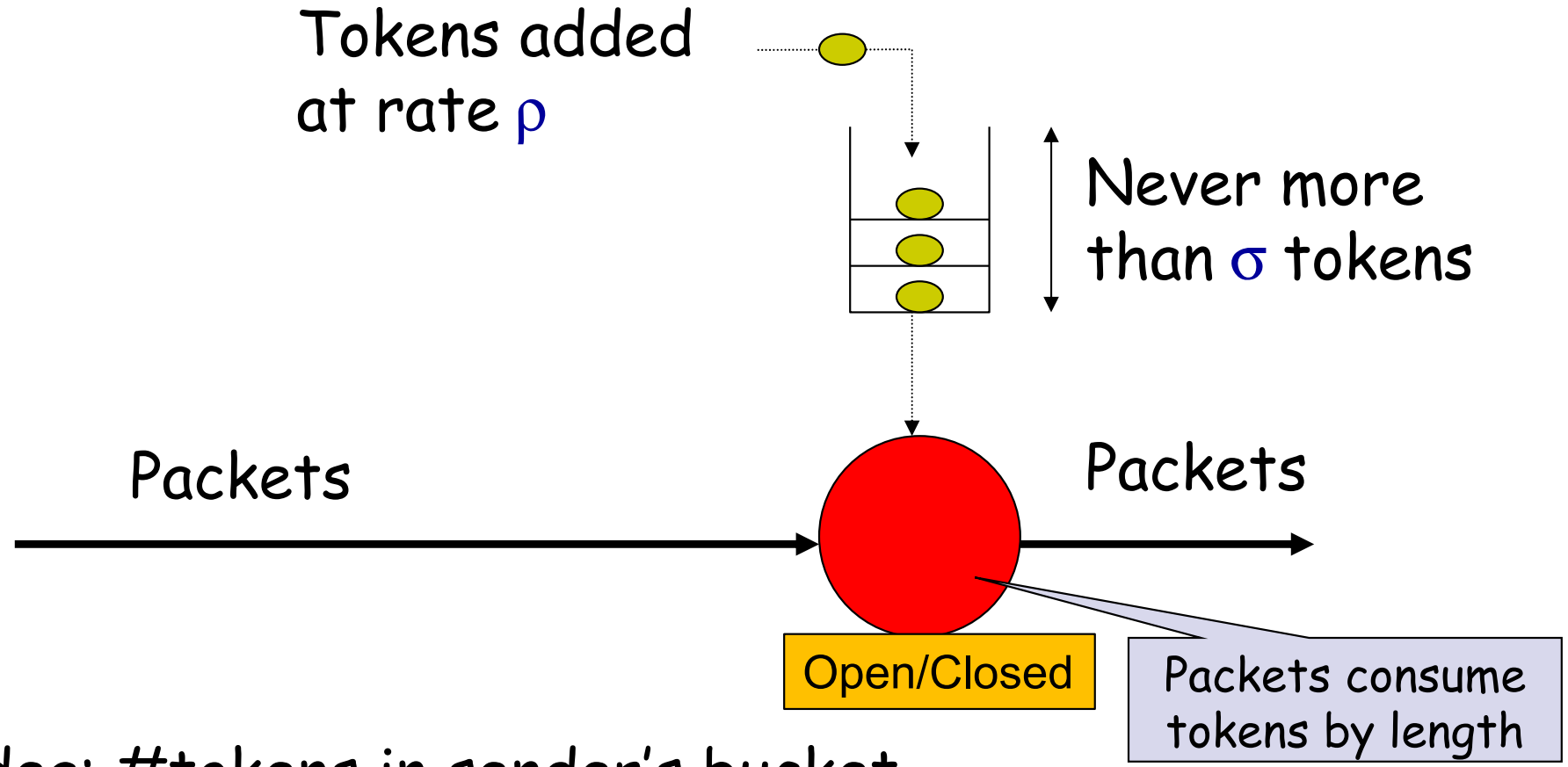
Def.: Given $\rho \geq 0$ and $\sigma \geq 0$, we write $R \sim (\rho, \sigma)$ iff for all times $y \geq x$, it holds:

$$B(x, y) \leq \sigma + \rho \cdot (y - x)$$

- If a stream is (σ, ρ)-constrained, then no bit is lost when the stream arrives at buffer with drain rate $\geq \rho$ and buffer size $\geq \sigma$.
- If a (σ, ρ)-constrained stream arrives in L-bit packets on a C-speed line, necessary to have

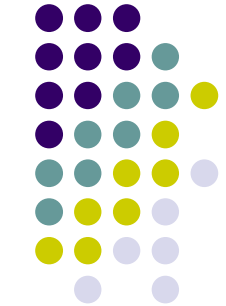
  $\sigma \geq \frac{L}{C}(C - \rho) = L\left(1 - \frac{\rho}{C}\right)$.

The "leaky bucket" regulator: packet release control



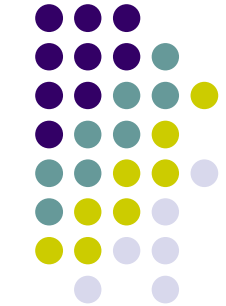
Idea: #tokens in sender's bucket
= #empty slots in receiver's buffer

Leaky Bucket Regulator



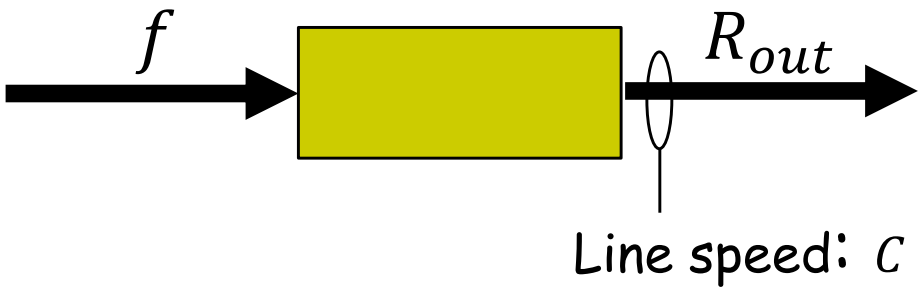
- The delay suffered by each packet is as small as possible, subject to the (σ, ρ) constraint
- Extension: **Dual** leaky bucket regulator. The stream has to conform to both (σ_1, ρ_1) and to (σ_2, ρ_2).
  - Used to describe rate within burst
  - Makes sense only when $\sigma_1 \leq \sigma_2$, $\rho_1 \geq \rho_2$ (w.l.o.g.)

Work-conserving elements



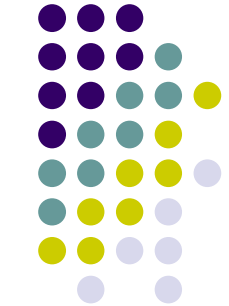
- An element with input flow f and output line speed C is work conserving if

$$W_\rho^f(t) \geq 0 \Rightarrow R_{out}(t) = C$$



Leaky bucket regulator is NOT work-conserving

Leaky Bucket Policing



Objective: Do not delay packets, only identify packets violating the (σ,ρ) envelope

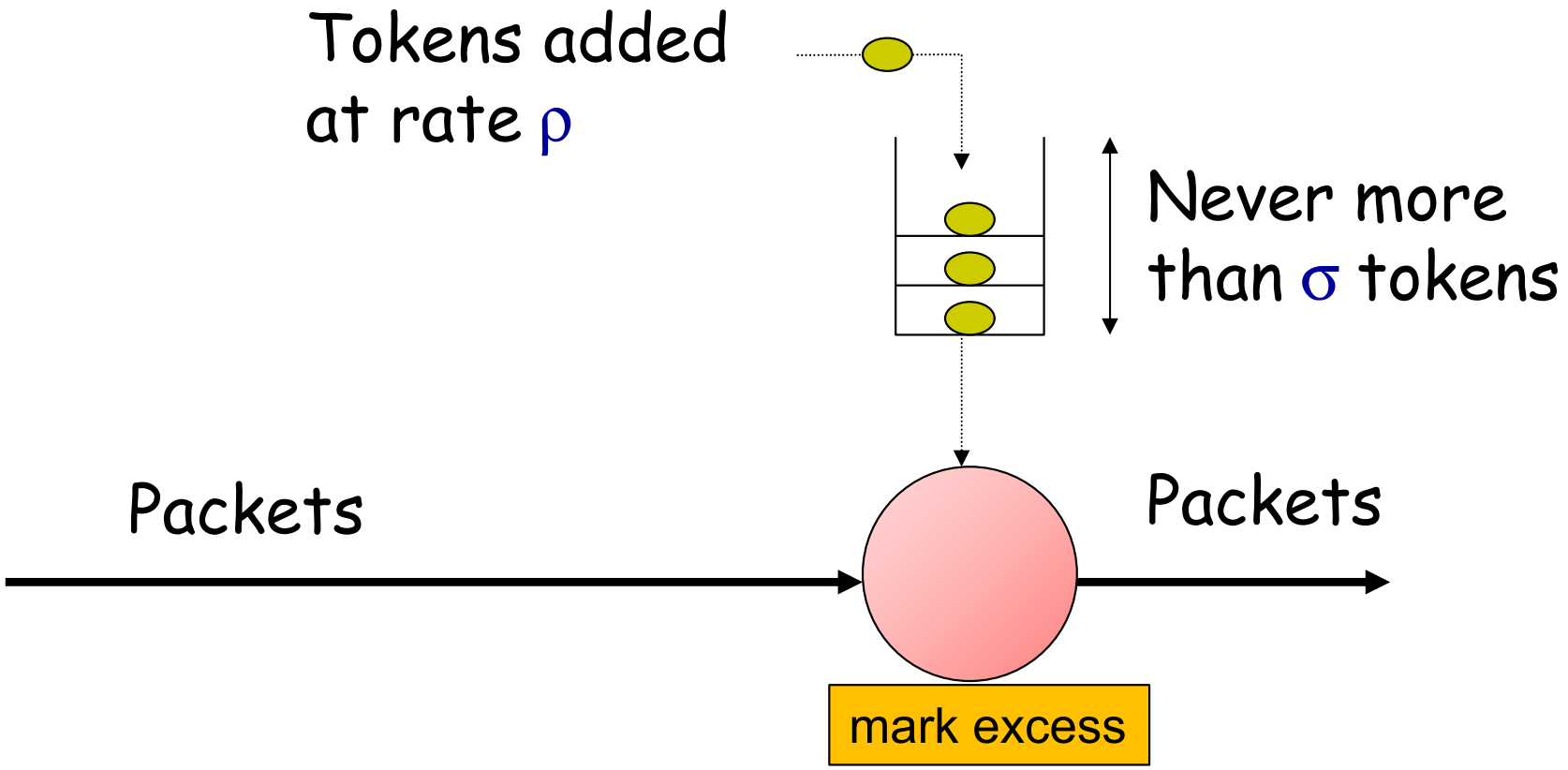
- packets are marked as "conforming" (green) or "excess" (red)

Excess packets are candidates to be dropped later

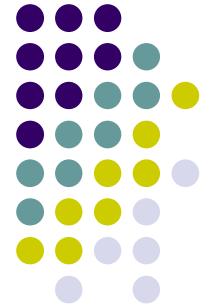
Rule: Maintain leaky bucket as before, except that packets are never delayed:

- If too few tokens, none is taken and packet is marked "excess"

Leaky bucket policing

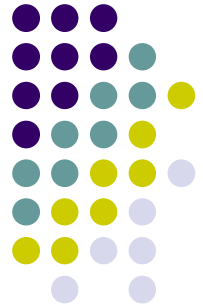


Some other network elements

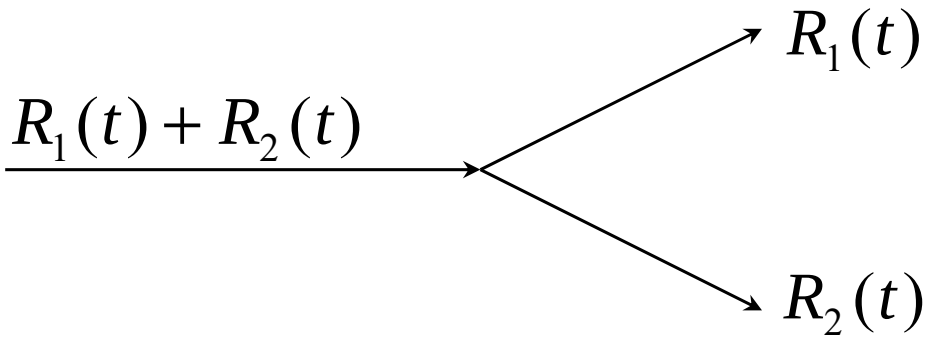


- Demultiplexer (switch input)
- Multiplexer (switch output)

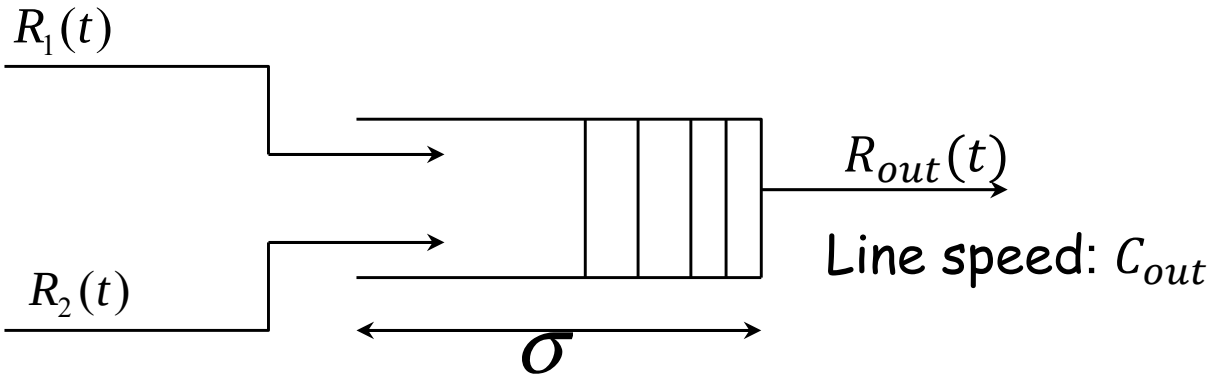
The Demultiplexer (DEMUX)



- Network element with a single input stream and two or more output streams.
- The DEMUX instantaneously determines the output substream.



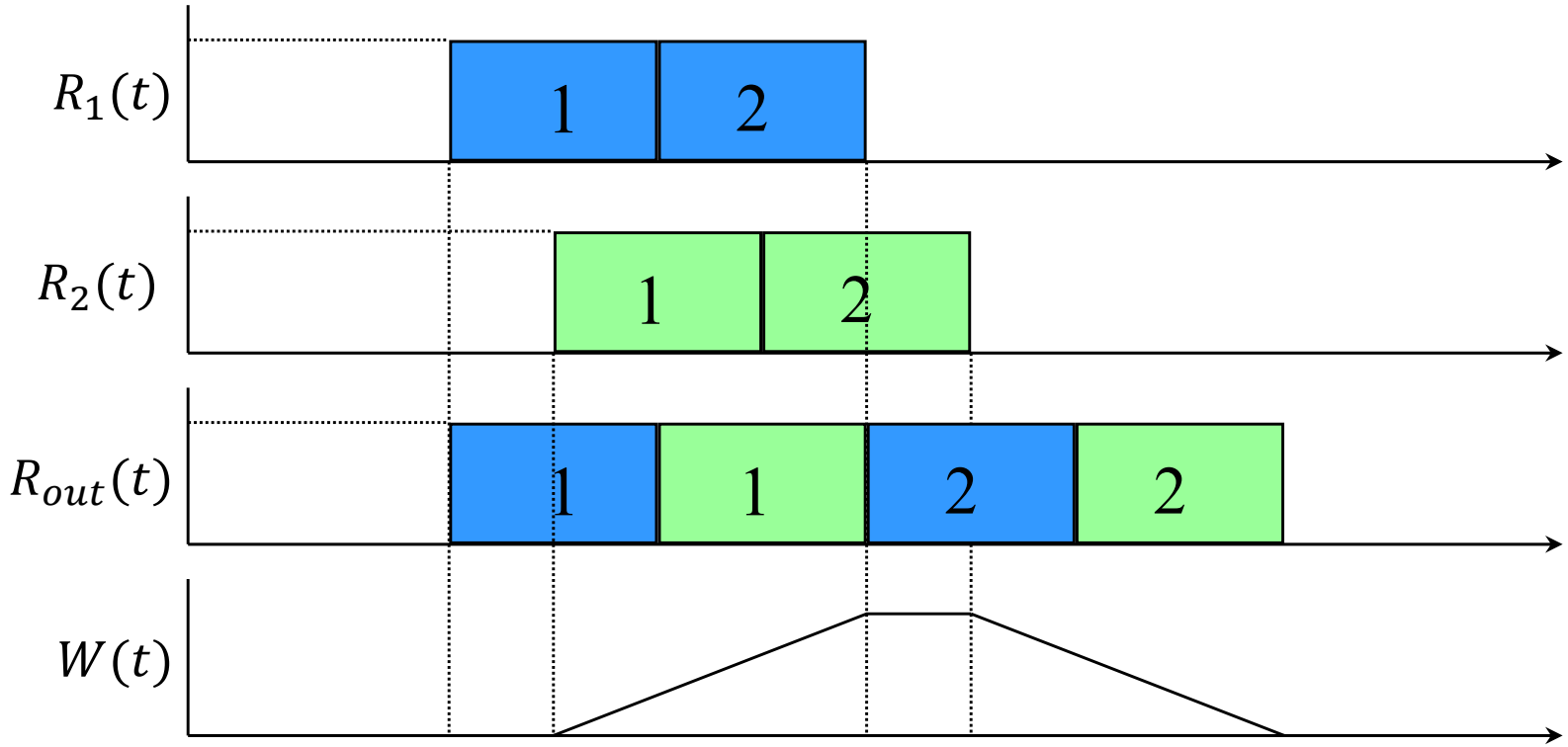
The Multiplexer (MUX)



- Element with 2 (or more) input streams with rates R_1, R_2 and a single output stream R_{out}
- Total input rate $R_1 + R_2$. If $R_i \sim (\sigma_i, \rho_i)$, and the mux is work-conserving, then the maximal delay is at most $\frac{\sigma_1+\sigma_2}{C_{out}-(\rho_1+\rho_2)}$

FCFS Work Conserving MUX

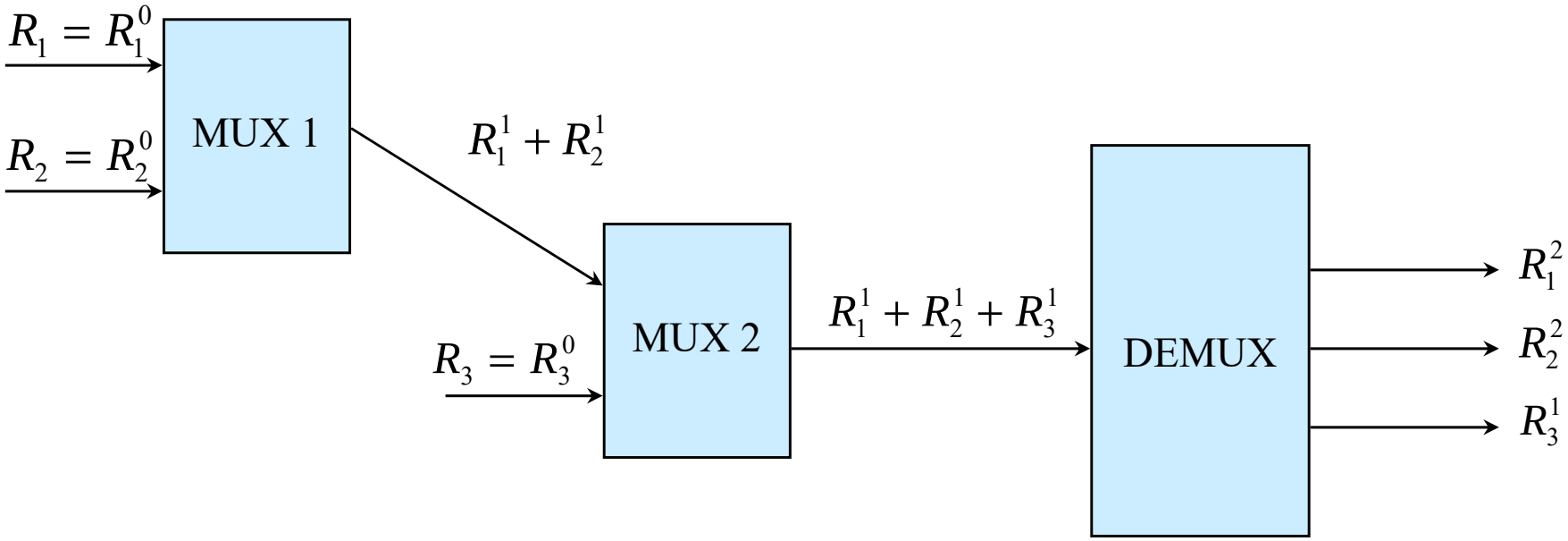
An example of First Come First Served MUX, output line rate = input line rate:



Delay in FCFS MUX

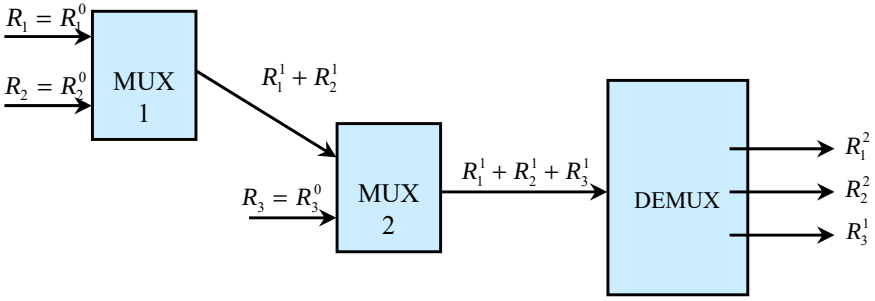
- The delay of a data bit from stream 1, which arrives at the multiplexer at time t, consists of:
  - The data within the MUX buffer at time t.
  - Up to L additional bits from stream 2 (something that is just starting to arrive)
- At Most: $d_1(t) \leq \frac{1}{C_{out}}\left(W_{C_{out}}^{R_1+R_2}(t) + L\right)$

Network Example



What happens to σ ?

Network Example



- Network properties:
  - For each i: $R_i \sim (\sigma_i, \rho_i)$
  - Work conserving MUXes
- The output of MUX1 is given by

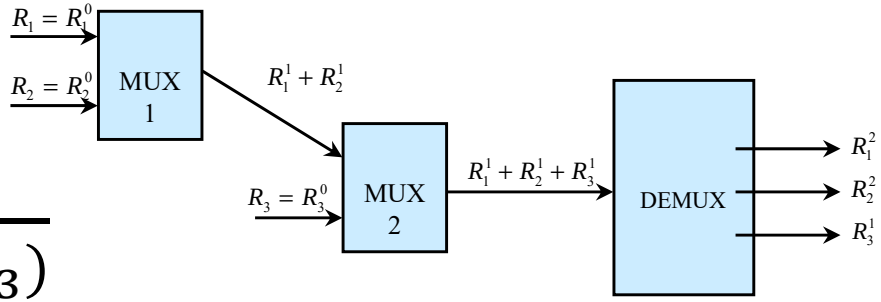
$$R_1^1 + R_2^1 \sim (\sigma_1 + \sigma_2, \rho_1 + \rho_2)$$

- The delay in MUX 1 is at most

$$D_1 = \frac{\sigma_1 + \sigma_2}{1 - \rho_1 - \rho_2}$$

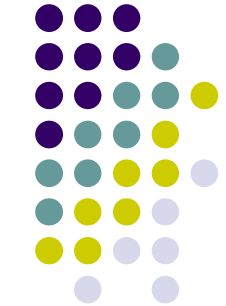
Network Example

$$D_2 = \frac{\sigma_1 + \sigma_2 + \sigma_3}{1 - \rho_1 - \rho_2 - \rho_3}$$



- The delay in MUX 2 is at most $D_2 = \frac{\sigma_1+\sigma_2+\sigma_3}{1-(\rho_1+\rho_2+\rho_3)}$
  The total network delay for sessions 1 and 2 is at most $D_1 + D_2$
- The total network delay for session 3 is at most D_2

New Concepts



- HoL blocking
- Input queuing
- Output queuing
- Combined Input-Output queuing
- Fabric speedup
- Stable marriage algorithm
- PIM, iSLIP alg's
- (σ, ρ)-constrained flow
- Leaky bucket regulator